

Phase-by-Phase Implementation Plan

Workflow Automation Platform — n8n-style Build

A complete, sequential execution plan — from initial setup to a scalable, SDK-ready production platform. Each phase lists its goal, the concrete tasks, the tech involved, and the exact criteria that define "done."

React + NestJS + TypeScript | 9 Phases | Build in Order, No Skipping

Prepared as a personal project reference • July 2026

How to Use This Plan

This plan is sequential by design — each phase depends on the one before it working correctly. Do not start Phase 3 while Phase 2 is half-working; an unstable foundation makes every phase after it harder to debug.

- Each phase has a clear **Goal**, a list of concrete **Tasks**, the **Tech Involved**, and a **"Done When"** checklist.
- Treat "Done When" as a hard gate — if you can't check every box, the phase isn't finished, even if it "mostly works."
- Update the master progress tracker (final page) as each phase completes.

Do not build the visual editor before Phase 5. The most common reason solo builders stall on projects like this is trying to build the UI and the execution engine at the same time. Get the engine working with raw JSON first.

Phase 0 — Planning & Project Setup

FOUNDATION • NO CODE LOGIC YET

Goal: Set up the repository structure, tooling, and environment so every later phase has a clean place to land — before writing any business logic.

Tasks

- 1. Initialize a monorepo using **pnpm workspaces** or **Turborepo**
- 2. Create the folder structure: /apps/frontend, /apps/backend, /apps/worker, /packages/shared-types, /packages/sdk
- 3. Set up TypeScript configs shared across all apps/packages
- 4. Set up ESLint + Prettier for consistent code style from day one
- 5. Initialize Git repository, write a basic README describing the project
- 6. Set up Docker Compose for local dev: PostgreSQL + Redis containers
- 7. Create a basic NestJS app skeleton (apps/backend) that boots and responds to a health-check route
- 8. Create a basic React app skeleton (apps/frontend) that boots and shows a placeholder page

Tech Involved

Tool	Purpose
pnpm / Turborepo	Monorepo package management and build orchestration
Docker Compose	Local PostgreSQL + Redis, no manual installs
NestJS CLI	Scaffold the backend app
Vite + React	Scaffold the frontend app

Done when: Repo is cloneable, docker compose up starts Postgres + Redis, backend health-check route returns 200, frontend loads a blank page — all with one setup command.

Phase 1 — Core Execution Engine (No UI)

THE HEART OF THE PLATFORM

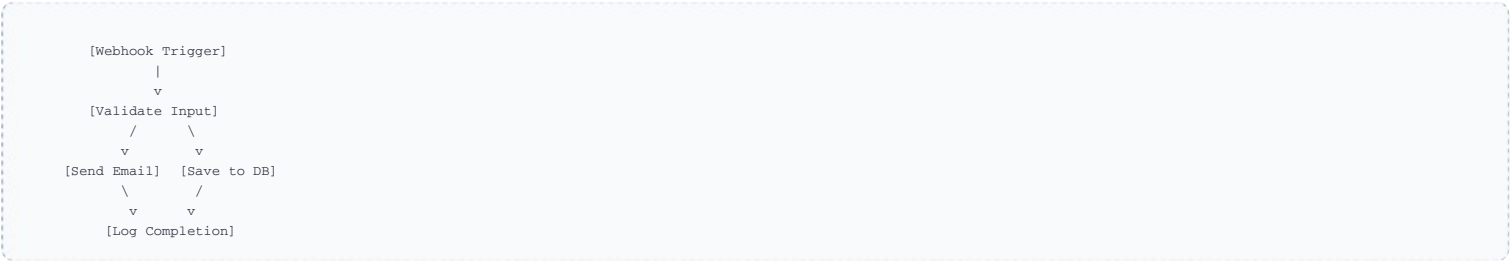
Goal: Build a working engine that can take a JSON-defined workflow, walk it as a DAG, and execute each node in order — entirely without a visual interface.

Tasks

- Design the **workflow JSON schema**: nodes array, edges array, each node has an id, type, and config
- Build the **Node Interface** — a contract every node type must implement (input schema, output schema, execute function)
- Implement 3-4 basic node types to prove the concept:
 - Webhook Trigger** — starts a run when hit
 - HTTP Request** — calls an external API
 - Set/Transform Data** — reshapes data between steps
 - Conditional (If)** — branches the flow
- Build the **DAG walker**: given the JSON graph, determine execution order (topological sort), respecting parallel branches
- Pass each node's output as input to its connected child node(s)
- Add basic error handling: if a node fails, halt that branch and log the failure
- Write 2-3 hardcoded example workflows (as JSON files) to test the engine end-to-end

Tech Involved

Concept	Notes
Directed Acyclic Graph (DAG)	Core data structure — nodes + directional edges, no cycles
Topological Sort	Algorithm used to determine valid execution order of nodes
NestJS Services	Encapsulate the engine logic as an injectable service



Done when: A hardcoded JSON workflow with a webhook trigger, an HTTP call, and a conditional branch executes correctly end-to-end from the command line, with output logged at each step — no database, no queue, no UI needed yet.

Phase 2 — Node System & Node Library

MAKING NODES PLUGGABLE

Goal: Turn the hardcoded node types from Phase 1 into a proper pluggable node system, and expand the library of available nodes.

Tasks

1. Formalize the **Node Registry** — a central place where all available node types are registered and looked up by type name
2. Refactor nodes so adding a new one doesn't require touching the core engine code
3. Add more node types: **Schedule/Cron Trigger**, **Send Email**, **Database Read/Write**, **Switch (multi-branch)**, **Delay/Wait**
4. Add basic input validation per node using schemas (e.g. via **Zod** or **class-validator**)
5. Write unit tests for each node type in isolation

Tech Involved

Tool	Purpose
Zod or class-validator	Runtime schema validation for node inputs/outputs
Jest	Unit testing individual nodes

```
interface NodeDefinition {
  type: string;
  inputSchema: JSONSchema;
  outputSchema: JSONSchema;
  execute(input, credentials): Promise<output>;
}
```

Done when: A new node type can be added by creating one new file and registering it — with zero changes to the DAG engine itself. At least 8 node types exist and are unit tested.

Phase 3 — Asynchronous Execution (Queue)

MAKING IT PRODUCTION-REALISTIC

Goal: Move workflow execution off the request thread and into a proper async, retryable, horizontally-scalable worker system.

Tasks

- 1. Set up **Redis** as the queue backend
- 2. Integrate **BullMQ** for job queueing
- 3. Split the app: apps/backend (API, receives triggers, pushes jobs) vs apps/worker (consumes jobs, runs the DAG engine from Phase 1-2)
- 4. Implement retry logic with exponential backoff for failed nodes
- 5. Implement a **dead-letter queue** for jobs that fail permanently after max retries
- 6. Add job status tracking: queued → running → completed / failed

Tech Involved

Tool	Purpose
Redis	In-memory store backing the job queue
BullMQ	Queue library — job scheduling, retries, concurrency control

Why this split matters: The API server stays fast and only handles requests/storage. Execution happens in separate worker processes, so more workers can be added under load without touching the API layer at all — this is the actual mechanism behind horizontal scaling.

Done when: Triggering a workflow returns immediately (job queued), a separate worker process picks it up and executes it, a deliberately-failing node retries automatically, and a permanently failing job lands in the dead-letter queue instead of disappearing.

Phase 4 — Persistence Layer

NOTHING IS TEMPORARY ANYMORE

Goal: Store workflow definitions, execution history, and logs permanently instead of relying on hardcoded JSON files.

Tasks

- 1. Set up **PostgreSQL** with an ORM (**Prisma** or **TypeORM**)
- 2. Design core tables: workflows, workflow_versions, executions, execution_logs, credentials
- 3. Replace hardcoded JSON workflow files with database-stored definitions
- 4. Build CRUD API endpoints in NestJS: create/read/update/delete workflows
- 5. Log every node execution (input, output, timestamp, status) tied to an execution record
- 6. Build an endpoint to fetch execution history for a given workflow

Tech Involved

Tool	Purpose
PostgreSQL	Primary relational database
Prisma / TypeORM	Type-safe database access from NestJS

Done when: A workflow can be created via API, saved to the database, triggered, and its full execution history (including per-node input/output) can be retrieved afterward — and restarting the server doesn't lose any data.

Phase 5 — Visual Canvas (Frontend)

ONLY NOW DOES THE UI BEGIN

Goal: Build the drag-drop visual editor on top of an engine that already works — the UI is now just a JSON generator/viewer for a proven backend.

Tasks

1. Integrate **React Flow** for the node canvas
2. Build a node palette/sidebar listing available node types (pulled from the backend's Node Registry)
3. Implement drag-drop node placement and edge connection on the canvas
4. Build per-node configuration panels (forms generated from each node's input schema)
5. Serialize the visual graph into the same JSON format the backend expects
6. Wire up "Save Workflow" and "Run Workflow" buttons to the backend API
7. Build an execution view: show real-time status per node (running/success/failed) using polling or WebSockets

Tech Involved

Tool	Purpose
React Flow	Node-graph canvas library — the standard choice for this exact use case
React Hook Form / Zod	Dynamic form generation from node schemas
WebSockets (Socket.IO) or polling	Real-time execution status updates on the canvas

Done when: A user can drag nodes onto a canvas, connect them, configure each one, save the workflow, trigger a run, and watch node-by-node execution status update live — entirely through the UI, with zero manual JSON editing required.

Phase 6 — Authentication, Credentials & Multi-Tenancy

MAKING IT SAFE FOR REAL USERS

Goal: Support multiple users/organizations securely, with properly isolated credentials and workflow ownership.

Tasks

1. Implement user authentication (JWT-based, or OAuth via a provider)
2. Add **NestJS Guards** to protect API routes by authenticated user
3. Implement **encrypted credential storage** — API keys/tokens never stored in plaintext
4. Scope credentials and workflows per user/organization (multi-tenancy)
5. Add role-based access control (owner, editor, viewer) if collaboration is planned
6. Add audit logging for sensitive actions (credential changes, workflow deletion)

Tech Involved

Tool	Purpose
Passport.js (NestJS integration)	Authentication strategy handling (JWT/OAuth)
bcrypt / argon2	Password hashing
AES encryption (node crypto)	Encrypting stored credentials at rest

Never embed raw credentials inside workflow JSON. Nodes should reference a credential by ID only; the engine resolves and decrypts it at execution time, scoped to the workflow owner.

Done when: Two separate test users can each create workflows and credentials that are fully invisible to one another, and no credential value is ever visible in plaintext in the database or in workflow JSON exports.

Phase 7 — Expanded Trigger System & Integrations

GROWING REAL-WORLD USEFULNESS

Goal: Expand beyond basic webhook/cron triggers and grow the node library toward genuinely useful third-party integrations.

Tasks

1. Add more trigger types: polling triggers (check an API every N minutes), file-watch triggers, manual/test triggers
2. Build 3–5 real integration nodes (e.g. Slack message, Google Sheets row append, Stripe event, Notion page create)
3. Standardize how integration nodes handle auth (OAuth flow vs API key) using the credential system from Phase 6
4. Add rate-limit handling and pagination helpers for API-heavy nodes
5. Add a "test this node" feature so users can dry-run a single node before running the full workflow

Done when: At least one OAuth-based integration (e.g. Google or Slack) and one API-key-based integration work end-to-end inside real workflows, including proper error messages when auth expires.

Phase 8 — Public SDK & Developer API

OPENING THE PLATFORM UP

Goal: Extract shared types into a public SDK package, enabling developers to build custom nodes or interact with the platform programmatically.

Tasks

- 1. Formalize packages/shared-types into a clean, documented set of TypeScript types
- 2. Build packages/sdk: a client library wrapping the backend's REST API (create/trigger workflows programmatically)
- 3. Document a "Custom Node" spec so third parties (or future you) can build new node types without touching core code
- 4. Publish the SDK internally first (private npm registry or workspace-linked) before considering public release
- 5. Write API documentation (OpenAPI/Swagger via NestJS's built-in support)

Tech Involved

Tool	Purpose
NestJS Swagger module	Auto-generates OpenAPI docs from existing decorators
tsup / tsc	Building the SDK package for distribution

Done when: A separate small script (outside the monorepo) can install the SDK package and successfully create + trigger a workflow using only the published SDK — no direct database or internal-code access required.

Phase 9 — Deployment, Scaling & Hardening

PRODUCTION READINESS

Goal: Move from local development to a deployed, monitored, horizontally-scalable production setup.

Tasks

1. Containerize each app (frontend, backend, worker) with production Dockerfiles
2. Set up CI/CD (GitHub Actions) for automated build/test/deploy
3. Deploy backend + worker to a container platform (e.g. Docker Compose on a VPS, or Kubernetes for larger scale)
4. Add structured logging and basic monitoring/alerting (execution failures, queue backlog size)
5. Load-test the worker layer and confirm horizontal scaling (adding more worker instances increases throughput)
6. Security pass: rate limiting on public endpoints, input sanitization, dependency audit

Done when: The platform runs in a deployed environment (not localhost), can survive a worker restart without losing queued jobs, and scaling worker replicas measurably increases how many workflows can run concurrently.

Master Progress Tracker

Phase	Status
Phase 0 - Planning & Project Setup	🚧 Not started
Phase 1 - Core Execution Engine (No UI)	🚧 Not started
Phase 2 - Node System & Node Library	🚧 Not started
Phase 3 - Asynchronous Execution (Queue)	🚧 Not started
Phase 4 - Persistence Layer	🚧 Not started
Phase 5 - Visual Canvas (Frontend)	🚧 Not started
Phase 6 - Auth, Credentials & Multi-Tenancy	🚧 Not started
Phase 7 - Expanded Triggers & Integrations	🚧 Not started
Phase 8 - Public SDK & Developer API	🚧 Not started
Phase 9 - Deployment, Scaling & Hardening	🚧 Not started

Reminder: Each phase's "Done When" checklist is a hard gate. Don't move to the next phase until the current one is genuinely complete — a shaky foundation compounds into much harder debugging two or three phases later.

This is the master implementation reference for the n8n-style automation platform. Update phase status as each stage completes. Paste this into any new chat to restore full project context.